

EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 6-IDA: IDA Pro — Read, Hack, and Patch a Static Variable and a GPIO Input (Raw `.bin`)

ARM Cortex-M33 • GDB • Ghidra

October 9, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

- Build the lesson project with `Release` and get both an `.elf` and a raw `.bin`
 - Dump the **ELF symbol map** with `arm-none-eabi-nm` and use it as ground truth
 - Load the raw `.bin` into IDA at `0x10000000`
 - Read how a **static local** keeps a fixed RAM address while a **regular local** is inlined to a constant
 - **Break at** `main` on live silicon, even though `main` moves between programs
 - **Hack a running target live** by editing a register in IDA's Registers widget
 - **Resolve the functions in the IDA GUI** using the ELF symbol map
 - **Patch** the value byte, invert the button/LED logic, and (optionally) rename the printed label
 - **Export** the patched image, convert it to UF2, and flash it
-

How This Guide Works

The build produces two files for the project:

File	What it is	How we use it
<code>.elf</code>	The linked image with a full symbol table	Ground truth for every function address and name
<code>.bin</code>	The raw flash image, no headers, no symbols	The image we load into IDA and reverse

The `.bin` is built **from** the `.elf`, so the ELF tells you exactly what is at every address. We use the ELF symbol map to resolve functions in IDA, and we reverse-engineer the raw `.bin` the way a real extracted firmware image is reversed.

Build Release , not Debug . Every address in this guide matches the Week 6 lesson, and the Week 6 lesson is a Release build. Release optimizes the code the same way the original lesson was built: it **inlines** the static helper `demo_static_variable` (and the SDK's `gpio_set_dir` / `gpio_get` / `gpio_put` / `gpio_pull_up` helpers) into `main` , and it folds the regular local `regular_fav_num` down to a bare `movs r1, #42` constant. If you build Debug , the SDK function addresses move and the helper stays a separate call, so nothing lines up. Always build Release for this lesson.

The order is **dynamic first, static second**:

1. Break on the live target and prove what the code does.
2. Hack it live in the debugger and watch the output change.
3. Resolve the functions in IDA using the ELF symbol map.
4. Patch the bytes, export, convert, and flash.

Project	Prints	Also does	The hacks
<code>0x0014_static-variables</code>	<code>regular_fav_num: 42</code> , then <code>static_fav_num: incrementing</code>	reads button GPIO 15, drives LED GPIO 16	change <code>42</code> to <code>43</code> , invert the button/LED logic, (optional) rename the label

Addresses come from your build. Every address here is from the Release build produced in Step 3. Confirm against your own .elf with the command in Step 4.

`static_fav_num` **is the star of this week.** `regular_fav_num` is an automatic (stack) variable that the compiler bakes into a constant; `static_fav_num` is a function-local static that the linker parks at a **fixed RAM address** (`0x200005a8`) in `.data` . That fixed address is why it persists across loop iterations, and why you can read and patch it like a global.

Part 1: Build, Flash, and Get the Symbol Map

Step 1: Install the toolchain

Windows x64

- Install the **Raspberry Pi Pico** extension in VS Code. It installs the ARM GNU toolchain, CMake, Ninja, and the Pico SDK.
- Install **IDA Pro** and complete its license activation.
- Install **PuTTY** for the serial monitor.

macOS Apple Silicon

```
brew install cmake ninja
```

- Install **IDA Pro** and complete its license activation.
- Install the **Arm GNU Toolchain**, or let the VS Code Pico extension manage it.

Linux x64

```
sudo apt install cmake ninja-build gcc-arm-none-eabi libnewlib-arm-none-eabi git python3 openocd minicom
```

- Install **IDA Pro** and complete its license activation.

Step 2: Verify your tools are the right architecture (do not skip this)

On **macOS Apple Silicon**, the most common failure is an Intel `x86_64` tool on your `PATH`:

```
zsh: bad CPU type in executable: cmake
```

You may have **two Homebrews**: the arm64 one at `/opt/homebrew` and the Intel one at `/usr/local`. If `/usr/local/bin` wins, every `brew` tool is `x86_64`. Check:

```
file "$(which cmake)"
file "$(which ninja)"
file "$(which arm-none-eabi-gdb)"
file "$(which arm-none-eabi-nm)"
file "$(which openocd)"
file "$(which telnet)"
```

All must report `arm64`. If any is `x86_64`, put the Apple Silicon prefix first for the session and check again:

```
export PATH="/opt/homebrew/bin:$PATH"
hash -r
file "$(which cmake)"
```

To make it permanent, add that `export` to `~/.zshrc`. Do not use Rosetta as a fix; OpenOCD and GDB are exactly the kind of programs where a translation layer produces failures that look like debugger bugs.

`telnet` **is special — and optional**. The GDB MI workflow does not need it; it is only used by the command-port fallback. macOS no longer ships `telnet`, and the Homebrew build is often the Intel one, so `telnet 127.0.0.1 4444` fails with `bad CPU type in executable`. Your `brew` command itself may also be the Intel one: if `brew install telnet` fails with `.../portable-ruby/.../ruby: Bad CPU type in executable`, you are running the Intel Homebrew. Call the Apple Silicon Homebrew explicitly:

```
/opt/homebrew/bin/brew install telnet
```

If you would rather not install anything, macOS ships an arm64 `nc`, which can connect to the same OpenOCD port:

```
nc 127.0.0.1 4444
```

Windows x64 and **Linux x64** do not have this problem. Skip to Step 3.

Step 3: Build the project with `Release`

Run this inside `0x0014_static-variables/`:

```
cmake -B build -G Ninja -DPICO_BOARD=pico2 -DPICO_PLATFORM=rp2350 -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Point IDA at this repository (once). Every console snippet below reads the repo root from `~/.embedded-hacking-repo`, so IDA never needs a database open and nothing is hardcoded. From the repo root, run once:

macOS / Linux:

```
pwd > ~/.embedded-hacking-repo
```

Windows (PowerShell):

```
(Get-Location).Path | Set-Content "$env:USERPROFILE\.embedded-hacking-repo"
```

Then build from the IDA console, so the whole build -> patch -> flash loop stays inside IDA. The console inherits a minimal `PATH` — on macOS just `/usr/bin:/bin:/usr/sbin:/sbin` — so it does not see Homebrew; add your package manager's `bin` first, then run plain `cmake`.

macOS Apple Silicon:

```
import os, subprocess
root = open(os.path.expanduser("~/embedded-hacking-repo")).read().strip()
os.environ["PATH"] = "/opt/homebrew/bin:" + os.environ["PATH"] # the console's PATH omits Homebrew
proj = os.path.join(root, "0x0014_static-variables")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
               "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/embedded-hacking-repo")).read().strip()
proj = os.path.join(root, "0x0014_static-variables")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
               "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/embedded-hacking-repo")).read().strip()
proj = os.path.join(root, "0x0014_static-variables")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
               "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

The build directory now contains the pair we need:

- `0x0014_static-variables/build/0x0014_static-variables.elf` and `.bin` — the `.bin` is **15516** bytes (`0x3c9c`)

If the ARM toolchain is not on your `PATH`, add `-DPICO_TOOLCHAIN_PATH=...`:

OS	Typical toolchain path
Windows x64	<code>C:/Program Files/Arm GNU Toolchain arm-none-eabi/14.2 rel1/bin</code>
macOS Apple Silicon	<code>~/pico-sdk/toolchain/14_2_Rel1/bin</code>
Linux x64	<code>/usr</code>

This guide's toolchain lives at `~/pico-sdk/toolchain/14_2_Rel1/bin`. All of `arm-none-eabi-nm`, `arm-none-eabi-objdump`, and `arm-none-eabi-gdb` resolved in this document come from there. If your install is elsewhere, which `arm-none-eabi-nm` tells you where to point.

Step 4: Dump the ELF symbol map

This is the ground truth for the whole lesson. Run `arm-none-eabi-nm` on the ELF and keep the output in a terminal or a text file:

macOS Apple Silicon / Linux x64:

```
arm-none-eabi-nm -n --defined-only build/0x0014_static-variables.elf | grep -E ' [Tt] '
```

Windows x64:

```
arm-none-eabi-nm -n --defined-only build\0x0014_static-variables.elf | Select-String ' [Tt] '
```

Each line is `address type name`. The `T / t` type is a function. Here are the functions this lesson uses. The signatures come from the ELF's DWARF debug info queried with `arm-none-eabi-gdb -batch -ex "ptype <name>"`, so they are exact.

Our code and the startup chain:

Address	ELF symbol	Signature	Role
0x1000015c	<code>_reset_handler</code>	<code>void _reset_handler(void)</code>	reset entry
0x10000186	<code>platform_entry</code>	<code>void platform_entry(void)</code>	calls <code>runtime_init</code> , <code>main</code> , <code>exit</code>
0x1000019a	<code>data_cpy</code>	<code>void data_cpy(void*, void*, void*)</code>	copies <code>.data</code> from flash to SRAM
0x100001e4	<code>_init</code>	<code>void _init(void)</code>	runs <code>.init_array</code>
0x10000210	<code>frame_dummy</code>	<code>void frame_dummy(void)</code>	C runtime boilerplate
0x10000234	<code>main</code>	<code>int main(void)</code>	the lesson function (<code>demo_static_variable</code> inlined)

The GPIO functions `main` calls (or that are inlined into it):

Address	ELF symbol	Signature	Role
0x1000029c	<code>gpio_set_function</code>	<code>void gpio_set_function(uint, gpio_function_t)</code>	SDK GPIO function select (UART pins)
0x100002d8	<code>gpio_set_pulls</code>	<code>void gpio_set_pulls(uint, bool, bool)</code>	SDK pull config (<code>gpio_pull_up</code> inlined)
0x10000300	<code>gpio_init</code>	<code>void gpio_init(uint)</code>	SDK GPIO init

The stdio/UART and printf chain `main` reaches:

Address	ELF symbol	Signature	Role
0x10000e60	time_us_64	uint64_t time_us_64(void)	SDK microsecond clock
0x10000ed0	uart_init	uint uart_init(uart_inst_t*, uint)	SDK UART init
0x10002dbc	exit	void exit(int)	C runtime exit
0x10002dc4	runtime_init	void runtime_init(void)	SDK runtime init
0x10002df0	stdio_out_chars_crlf	void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)	CRLF output driver
0x10002f00	stdio_put_string	int stdio_put_string(const char*, int, bool, bool)	buffered string output
0x10002fec	stdio_set_driver_enabled	void stdio_set_driver_enabled(stdio_driver_t*, bool)	enable the UART driver
0x10003014	stdio_init_all	bool stdio_init_all(void)	SDK serial init
0x100030e0	__wrap_vprintf	int __wrap_vprintf(const char*, va_list)	printf core
0x100031a4	__wrap_printf	int __wrap_printf(const char*, ...)	the printf wrapper
0x10003360	stdio_uart_init	void stdio_uart_init(void)	SDK UART stdio init
0x100034a0	strlen	size_t strlen(const char*)	C runtime string length

SDK helpers the `printf` and `uart_init` paths reach:

Address	ELF symbol	Signature	Role
0x10002d60	vfctprintf	int vfctprintf(void (*)(char, void*), void*, const char*, va_list)	printf format engine
0x1000237c	_vsprintf	int _vsprintf(out_fct_type, char*, size_t, const char*, va_list)	the format dispatcher
0x10001760	_ntoa_format	unsigned _ntoa_format(out_fct_type, char*, size_t, size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)	number formatter
0x100016c4	_out_rev	unsigned _out_rev(out_fct_type, char*, size_t, size_t, const char*, size_t, unsigned, unsigned)	reversed-digit output
0x10001934	_out_char	void _out_char(char, void*, size_t, size_t)	single-char sink
0x10000e74	busy_wait_us	void busy_wait_us(uint64_t)	UART timing loop

Address	ELF symbol	Signature	Role
0x100010a4	clock_get_hz	unsigned long clock_get_hz(clock_handle_t)	UART clock lookup

Two things in this project have **no symbol of their own**, because the compiler inlined them into `main`:

- `demo_static_variable` — the `static` helper in our own source is inlined, so there is no `demo_static_variable` address to rename. You see its body directly inside `main`.
- `init_gpio`, `gpio_set_dir`, `gpio_get`, `gpio_put`, and `gpio_pull_up` — these are `static inline` in the SDK headers, so they compile to the `mcr` / `SIO` writes and the `ldr` / `ubfx` reads you see in `main` rather than to calls. `gpio_pull_up(15)` becomes the direct call to `gpio_set_pulls` at `0x1000024e`.

`static_fav_num` is a `t` **symbol, not a function**. `arm-none-eabi-nm -n` lists `200005a8 t static_fav_num.0`. That lowercase `t` is a **local data** symbol: the linker named the function-local static `static_fav_num.0` and parked it at RAM address `0x200005a8`. There is no pool entry for `regular_fav_num` because it is an automatic (stack) variable — and here the compiler never gave it a stack slot at all, folding it into a constant.

Step 5: Flash and confirm the output

A `.bin` has no headers, so OpenOCD must be told the base address `0x10000000`. From the repository root:

macOS Apple Silicon / Linux x64:

```
./flash.sh 0x0014_static-variables/build/0x0014_static-variables.bin
```

Windows x64 (PowerShell):

```
.\flash.ps1 -Bin 0x0014_static-variables\build\0x0014_static-variables.bin
```

Or flash from the IDA console (the console reads the repo root from the marker file, so it works with no database open):

macOS Apple Silicon / Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/embedded-hacking-repo")).read().strip() # set once (Step 3)
bin_path = os.path.join(root, "0x0014_static-variables", "build", "0x0014_static-variables.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"]) # free the probe first
subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/embedded-hacking-repo")).read().strip() # set once (Step 3)
bin_path = os.path.join(root, "0x0014_static-variables", "build", "0x0014_static-variables.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"]) # free the probe first
subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                 os.path.join(root, "flash.ps1"), "-Bin", bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("flashing in the background; log:", log)
```

Wait for `wrote 15516 bytes ...` and `** Verified OK **`. Open a serial monitor at **115200** baud:

- **Windows x64:** PuTTY -> Connection type **Serial**, the Pico's COM port, speed `115200`.
- **macOS Apple Silicon:** `screen /dev/tty.usbmodem* 115200` (quit with `Ctrl-A` then `K`).
- **Linux x64:** `minicom -D /dev/ttyACM0 -b 115200`.

```
regular_fav_num: 42
static_fav_num: 42
regular_fav_num: 42
static_fav_num: 43
regular_fav_num: 42
static_fav_num: 44
...
```

`regular_fav_num` stays at `42` every pass (it is recreated as `42` each loop), while `static_fav_num` keeps climbing (it persists at `0x200005a8`). Wire a push button from GPIO 15 to GND and an LED from GPIO 16 (through a resistor) to GND. With the stock image, the LED is **off** when the button is released and **on** when you press it — the pull-up holds GPIO 15 high when released, and the code inverts that with `eor.w r3, r3, #1`. We will flip that behavior in Step 18b.

Part 2: Load the Raw `.bin` into IDA

Start from a fresh IDA state. If you already have a database for this lesson, **close it and start over**; a stale database keeps old names and patches.

Step 7: Bring the raw `.bin` into IDA

A raw `.bin` has no headers, so IDA cannot know where it belongs or what architecture it is. You must supply both. If you just double-click the `.bin`, IDA may load it with a guessed architecture, and every address in this lesson will be wrong.

1.1 Open the file

1. **File** ▶ **Open...** (Ctrl/Cmd+O).
2. Select the `.bin` file and click **Open**.

IDA opens the "**Load a new file**" dialog because a raw `.bin` has no header.

1.2 Set the format and processor

In the *Load a new file* dialog:

1. **Format:** leave it as **Binary file** (IDA auto-detects a headerless image).
2. **Processor type:** click the `...` button next to the processor field. In the *Processor type* dialog:
 - Family: **ARM**
 - Type: **ARM Little-endian** (the 32-bit ARM module).
 - Click **OK**.

If IDA asks "**Do you want to disassemble it as 64-bit code?**" — click **No**. The RP2350 is 32-bit; answering **Yes** loads AArch64 and you get `x` registers / data. (This is the single most common cause of the `DCB wall`.)

1.3 Configure the ARM architecture and Thumb mode

Still in the *Load a new file* dialog, open the processor options (the **Processor options** / **Set...** button, which opens the dialog titled **ARM architecture options**):

- **ARM architecture:** choose **ARMv8-M** (this is the Cortex-M33 / RP2350 core).
- **Thumb instructions:** select **Thumb-2** (ARMv8-M is Thumb-only, so this is the correct mode).
- Leave **automatic ARM-THUMB switch** off; we want Thumb throughout.

1.4 Set the loading address

- **Loading address:** enter `10000000` (hex; the RP2350 XIP flash base). IDA rejects addresses that don't look like ROM/RAM with *"The loading address should belong to RAM or ROM"* — `0x10000000` is the correct flash window.

Leave **Manual load** and **Create segments** at their defaults. Click **OK**.

1.5 Force Thumb, then seed the code from the vector table

A raw `.bin` has no header, so IDA's loader has **no entry point** and defaults the segment to **ARM mode** (which cannot decode Thumb). Two required steps — in this order:

A. Set the segment to Thumb (do this first).

- **Edit ▶ Segments ▶ Set default segment register value...** → register `T` = `1` (or press `Alt+G`). IDA reanalyzes the segment as Thumb automatically.

B. Seed the code from the vector table.

1. `G` → `10000000`. The first two words are the initial stack pointer and the **reset vector**.
2. **Read the 4-byte word at** `0x10000004` (do **not** assume its value — it differs per build).
3. **Clear bit 0** (the ARM **Thumb** flag) → the reset handler address.
4. `G` to that address → press **C (MakeCode)**. IDA disassembles the reset handler and follows its `bl` s into `main`.

Once `main` shows code, the import is correct.

Why IDA needs this and others don't: IDA's generic raw-binary ARM loader treats the image as flat data and waits for you to set Thumb and mark the entry point. It's an IDA quirk, not a mistake on your part.

Step 8: The views you will use

- **Linear view:** the disassembly listing. You navigate, read, and patch here.
- **Graph view:** the control-flow graph of the current function.
- **Decompiler (HLIL):** the pseudo-C decompilation.
- **Hex view:** raw bytes, used for patching.
- **Function list:** the sidebar list of every detected function.

Navigation: `G` go to address, `N` rename, `Y` set type or signature, `;` add a comment. Breakpoints are set from the GUI through the GDB MI adapter — see Step 12.

macOS function keys: the top-row `F` keys are usually mapped to system functions. Every step here uses menu paths that work without them.

Part 3: Dynamic — Break at `main` and Hack Live

Step 9: Start OpenOCD as a live debug server

Make sure no other OpenOCD is running; a forgotten server holds port `3333`.

macOS / Linux:

```
ps aux | grep -i openocd
```

Windows (PowerShell):

```
Get-Process | Where-Object { $_.ProcessName -like '*openocd*' }
```

Stop any leftover server gracefully:

macOS / Linux:

```
kill -TERM -f openocd
```

Windows (PowerShell):

```
Get-Process openocd -ErrorAction SilentlyContinue | Stop-Process
```

Start the server **parked at** `main`:

macOS Apple Silicon / Linux x64:

```
BP_ADDR=0x10000234 ./debug-server.sh
```

Windows x64 (PowerShell):

```
$env:BP_ADDR="0x10000234"; .\debug-server.ps1
```

Or start it from the IDA console, freeing the probe first and launching the server in the background so the console returns immediately:

macOS Apple Silicon / Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/embedded-hacking-repo")).read().strip() # set once (Step 3)
subprocess.run(["kill", "-TERM", "-f", "openocd"]) # stop any running server first
log = os.path.join(root, "openocd.log")
p = subprocess.Popen([os.path.join(root, "debug-server.sh")], cwd=root,
                     env=dict(os.environ, BP_ADDR="0x10000234"),
                     stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("OpenOCD started (pid", p.pid, "); log:", log)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/embedded-hacking-repo")).read().strip() # set once (Step 3)
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"]) # stop any running server first
log = os.path.join(root, "openocd.log")
p = subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
    os.path.join(root, "debug-server.ps1")], cwd=root,
    env=dict(os.environ, BP_ADDR="0x10000234"),
    stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("OpenOCD started (pid", p.pid, "); log:", log)
```

`Popen` returns in a few milliseconds; the server keeps running in the background. Check `openocd.log` for Listening on port 3333, then connect in Step 10.

Wait for:

```
Info : [rp2350.dap.core0] Examination succeed
Startup breakpoint at 0x10000234 (2-byte hardware execute, one-shot).
Info : starting gdb server for rp2350.dap.core0 on 3333
Info : Listening on port 3333 for gdb connections
```

BP_ADDR parks the core at main before any client connects. The script arms a 2-byte hardware breakpoint and then does the startup `reset run`, so the core runs from the vector table and stops at your address with no debugger attached yet. When IDA connects a moment later, the first thing it reads is already the truth: Stopped at `0x10000234`. This is the whole reason the lab works cleanly — you never have to drive a reset from outside the GUI.

Use the address you actually want to stop at:

What you want to stop at	Address	Command
main (once per reset)	0x10000234	BP_ADDR=0x10000234 ./debug-server.sh
The loop — the first <code>printf</code> call, hit every iteration	0x10000268	BP_ADDR=0x10000268 ./debug-server.sh

```
BP_ADDR=0x10000234 ./debug-server.sh # park at main
BP_ADDR=0x10000268 ./debug-server.sh # park in the loop instead
```

```
$env:BP_ADDR="0x10000234"; .\debug-server.ps1 # park at main
$env:BP_ADDR="0x10000268"; .\debug-server.ps1 # park in the loop
```

This startup stop is single-use. OpenOCD flushes breakpoints when a client connects, so this one is gone once IDA attaches — fine for `main`, which only runs once per reset. Every breakpoint after that is set from the IDA GUI (Step 12) and is repeatable. To stop at `main` again, restart the server with `BP_ADDR` and reconnect.

Exactly one core. The line must say `core0` and must **not** mention `core1`. `Core1` is never started by this firmware; exposing it makes IDA read `core1`'s reset-state registers, which are not real addresses, and OpenOCD floods the log with `Failed to read memory at 0xf0000000`. The scripts already use `USE_CORE=0`; do not change it.

Windows driver note: the Debug Probe must use the **WinUSB** driver. If OpenOCD reports `unable to open CMSIS-DAP device`, install it with [Zadig](#) (select Debug Probe (CMSIS-DAP) -> WinUSB).

Step 10: Connect IDA to the GDB server

1. Make sure the image is open and analyzed (Part 2) and the server from Step 9 is running (parked at `main`).
2. Choose **Debugger -> Connect to Remote Process**.
3. In the **adapter** dropdown, select **GDB MI**.
4. In the **connect** settings group, set **IP Address** to `127.0.0.1` and **Port** to `3333`.
5. Set **Full GDB Executable Path** to the `arm-none-eabi-gdb` from the **Arm GNU Toolchain 14.2.rel1**. It ships for all three hosts, and the Raspberry Pi Pico VS Code extension installs that same 14.2.rel1 toolchain (including `arm-none-eabi-gdb`) on all of them:

OS	arm-none-eabi-gdb path
macOS Apple Silicon	<code>/Applications/ArmGNUToolchain/14.2.rel1/arm-none-eabi/bin/arm-none-eabi-gdb</code> (or the Pico extension's <code>~/pico-sdk/toolchain/14_2_Rel1/bin/arm-none-eabi-gdb</code>)
Windows x64	<code>%USERPROFILE%\pico-sdk\toolchain\14_2_Rel1\bin\arm-none-eabi-gdb.exe</code> (Pico extension), or <code>C:\Program Files (x86)\Arm GNU Toolchain arm-none-eabi\14.2.rel1\bin\arm-none-eabi-gdb.exe</code>
Linux x64	<code>~/pico-sdk/toolchain/14_2_Rel1/bin/arm-none-eabi-gdb</code> (Pico extension), or the <code>bin/</code> directory of the extracted <code>arm-gnu-toolchain-14.2.rel1-x86_64-arm-none-eabi</code> tarball

6. Click **Accept**.

Use the GDB MI adapter. It launches a real `arm-none-eabi-gdb --interpreter=mi2` and lets IDA drive it, so breakpoints and stepping go through real GDB — which sends the correct 2-byte breakpoint length and handles step-over itself. Verified working end to end: connect, GUI breakpoints (**Add Hardware Breakpoint...**, hardware execute), **Step Into / Step Over**, and register edits. Stops are reported as `Breakpoint` (not `SingleStep`).

Do NOT have any breakpoints set in IDA before you connect. With the GDB MI adapter, attaching while IDA already has a breakpoint **hangs the session**. Start the server parked with `BP_ADDR` (Step 9), connect, and only add hardware breakpoints *after* the connection is up. This is a IDA bug; it is the single most common GDB MI failure.

The GDB executable path matters. Use the **14.2.rel1** build on every OS (Windows, macOS, Linux). The 13.3.rel1 build did **not** connect in testing.

This step is temporary. Vector35 plans to ship a GDB binary with the GDB MI adapter ([Vector35/debugger#929](#), milestone *Langara*). Once that lands, IDA provides GDB itself and you will not need to set **Full GDB Executable Path** at all.

Do not pick Corellium. IDA's adapter dropdown also lists **Corellium**, which is for Corellium's virtual devices and expects an API token, not a local OpenOCD server. It is not the adapter for this lab. The dropdown is a combo box, so an accidental arrow-key press can land on it — always read the label back and confirm it says **GDB MI** before clicking **Accept**.

The adapter and port are not saved in the .i64. Every time you relaunch IDA you must re-select **GDB MI**, re-enter port `3333`, and re-set the GDB path.

Watch for an off-screen error dialog. When a connection fails, IDA pops a `IDA critical alert` window that can be positioned mostly outside the main window, which makes it look like nothing happened. If the connect seems to do nothing, check your other display.

The target keeps running. Open the **Registers** tab (bug icon) and confirm you see live values. `pc` inside `0x10003xxx` and `sp` just below `0x20082000` are healthy.

If `pc` is `0x00000088`, `0x000000ec`, or `sp` is `0xf0000000`, the session is bad. Restart the server, then restart IDA (a server restart while attached leaves IDA in a stale session), and connect again.

Step 11: Find `main` without relying on its address

`main` can move between programs, so we do not guess it. We follow the one fixed path to it. Press `G` and go to `0x10000000`:

```
0x10000000  0x20082000  initial stack pointer (top of SRAM)
0x10000004  0x1000015d  reset vector
```

Bit 0 of a vector is the Thumb bit, so `0x1000015d` means "start at `0x1000015c`". That is `_reset_handler`. Follow the reset path to `0x10000186`, `platform_entry`:

```
10000186: 4914    ldr r1, [pc, #80] ; @ 0x100001d8
10000188: 4788    blx r1           ; runtime_init
1000018a: 4914    ldr r1, [pc, #80] ; @ 0x100001dc
1000018c: 4788    blx r1           ; main <-- the fixed anchor
1000018e: 4914    ldr r1, [pc, #80] ; @ 0x100001e0
10000190: 4788    blx r1           ; exit
10000192: be00    bkpt 0x0000
10000194: e7fd    b.n 0x10000192
```

The middle `blx` at `0x1000018c` is the call to `main`. `platform_entry` is byte-identical in every project, so `0x1000018c` catches `main` no matter where the linker placed it. The literal pool at `0x100001dc` holds `main | 1`; clearing bit 0 gives `0x10000234`.

Step 12: Set a hardware breakpoint in the GUI

With the **GDB MI** adapter, IDA sets breakpoints through real GDB, which sends the correct 2-byte length, so you set them **in the UI**. There is no command port here.

Why older drafts used the command port. IDA's **GDB RSP** adapter is its own minimal RSP client and sends a **1-byte** breakpoint (`Z0,<addr>,1`); the Cortex-M33 FPB comparators need 2 bytes, so OpenOCD rejected it with only breakpoints of two bytes length supported. The old workaround was to arm breakpoints by hand over telnet. **The GDB MI adapter does not have this problem** — it drives real `arm-none-eabi-gdb`, which sends the right length. So everything below is done in the GUI. The command port still exists as a fallback (see the end of this step), but you do not need it.

Where you can stop

You want to stop at	Address	How	Repeatable?
<code>main</code>	<code>0x10000234</code>	The server starts parked there with <code>BP_ADDR=0x10000234</code> (Step 9), so IDA is already stopped at <code>main</code> when it connects.	No — <code>main</code> runs once per reset.

You want to stop at	Address	How	Repeatable?
The regular <code>printf</code> call	0x10000268	Set a hardware breakpoint in the GUI, then click Resume .	Yes — fires on every iteration.
The static <code>printf</code> call	0x10000270	Set a hardware breakpoint in the GUI, then click Resume .	Yes — fires on every iteration.

Set the loop breakpoint in the GUI

1. Press **G**, type the loop address (`0x10000268`), and press Enter.
2. Set a **hardware execution** breakpoint at that address, either way:
 - `Debugger -> Add Hardware Breakpoint...` — a **hardware execute** (**HE**) breakpoint. **Use this one.**
 - click the line and press **F2** (`Debugger -> Toggle Breakpoint`) — a **software** breakpoint. It will **not** work here: the code is in read-only flash, so GDB cannot install it and the core just keeps running.
3. Click **Resume**. The core is already running the loop, so the breakpoint fires on the next iteration. IDA stops with the PC at the loop address and reports it as a **Breakpoint** — verified: `Stopped (Breakpoint) at 0x10000268`.

No breakpoints before you connect. With GDB MI, a breakpoint set before the connection hangs the session (Step 10). Start parked with `BP_ADDR`, connect, *then* add breakpoints.

Stepping

With the target halted at the breakpoint, **Step Into** (**F7**) and **Step Over** (**F8**) run through real GDB and move the PC. Verified: `0x10000268 -> 0x100031a4 -> 0x100031a6 -> ...`.

Step Over on the raw .bin steps into calls. The raw image has no symbol for `__wrap_printf`, so **Step Over** at the `printf` call behaves like **Step Into**. When the lab needs to execute the call and then stop, it moves the breakpoint to the return site and clicks **Resume** instead (Step 13 shows this).

Never use IDA's Restart button. On RP2350 it resets and halts inside the boot ROM (`pc=0x88`, `sp=0xf0000000`). To reset cleanly, restart the server with `BP_ADDR` and reconnect.

If you ever need the command port. It is still there — `nc 127.0.0.1 4444`, and `bp <addr> 2 hw` still arms a breakpoint, `rbp <addr> / rbp all` still remove them. It is the fallback if you switch back to the **GDB RSP** adapter, whose 1-byte breakpoints the GUI cannot set. With GDB MI you do not need it for this lab.

Step 13: HACK IT LIVE — change the printed value

`main` loads the constant `0x2a` (42) into `r1` and calls `printf` for the regular variable on every iteration. We break on that call and change it live:

1. Press `G`, go to `0x10000268` (the first `bl __wrap_printf`).
2. Set a **hardware execute** breakpoint there: Debugger → Add Hardware Breakpoint... (Do not use `F2` — that is a software breakpoint and will not work on read-only flash.)
3. Click **Resume** in IDA. The target is already running the loop, so the breakpoint fires on the next iteration. IDA stops with the program counter at `0x10000268` and `r1 = 0x2a`.
4. Open the **Registers** widget (bug icon → **Registers**).
5. Find `r1`. Its value is `0x2a` (42), loaded by the `movs r1, #42` at `0x10000264`.
6. **Set `r1` to `0x2b` (43)**. From IDA's Python console (Plugins → Python Console):

```
dbg.set_reg_value("r1", 0x2b)
```

`dbg.set_reg_value(name, value)` writes one register (returns `True` on success). You can also right-click `r1` in the **Registers** widget, press `E` (edit), type `2b`, and press Enter. The widget may not repaint the value, but the write reaches the target — you confirm it by the printed output in the next steps.

7. **Move the breakpoint past the call.** You want `printf` to run once and then stop, so move the breakpoint from `0x10000268` to the instruction *after* the call, `0x1000026c` (the `ldrb r1, [r4]` that begins the static-variable half of the loop): remove the breakpoint at `0x10000268` and set a hardware breakpoint at `0x1000026c`. Two reasons not to just click **Step Over** here: a breakpoint left on the current PC re-traps the step, and IDA's **Step Over** steps *into* `__wrap_printf` on this raw `.bin` because the image carries no symbol for the call. Moving the breakpoint to the return site is deterministic.
8. Click **Resume** in IDA. The core executes `bl __wrap_printf` with `r1 = 0x2b`, so this iteration prints `regular_fav_num: 43`, then stops at `0x1000026c`.
9. Look at your serial monitor — the `screen` session on the Pico's USB serial port — and at the **Target** tab in IDA:

```
regular_fav_num: 43
```

You changed a running program's output without touching the binary.

Step 13b: HACK THE STRING LIVE — change `regular_fav_num: to patched_fav_num:` (optional)

The text `"regular_fav_num: %d\r\n"` lives in flash (`.rodata`) at `0x10003560` , and flash is **read-only at runtime** — a debugger write there does not stick. So instead of overwriting the text in place, redirect the pointer: at the `printf` call, `r0` holds the string address, so point `r0` at a replacement string you place in RAM.

1. Arm the breakpoint at the `printf` call and hit it, exactly as in Step 13 steps 1-3. At the stop, `r0 = 0x10003560` and `r1 = 0x2a` .
2. Put the replacement string into free RAM at `0x20080000` from IDA's **Python console** (`Plugins -> Python Console`) — no command port needed:

```
dbg.write_memory(0x20080000, b"patched_fav_num: %d\r\n\x00")
```

`dbg.write_memory(address, bytes)` is IDA's debugger memory-write API; it returns `True` on success. That writes `patched_fav_num: %d\r\n\0` .

3. Point `r0` at that string:

```
dbg.set_reg_value("r0", 0x20080000)
```

(Or right-click `r0` in the **Registers** widget, press `E` , type `0x20080000` , and press Enter.)

4. If you want the value hack too, set `r1` to `0x2b` as in Step 13. Then move the breakpoint past the call in the GUI (remove it at `0x1000268` , set one at `0x100026c`) and click **Resume**. The core runs `printf` with `r0` pointing at your RAM string and `r1 = 0x2b` , so this iteration prints:

```
patched_fav_num: 43
```

then stops at `0x100026c` .

Like the value hack, this is **one iteration only**: the loop reloads `r0` (and `r1`) from flash on every pass, so the next line is `regular_fav_num: 42` again. The permanent version is the static patch in Step 18c.

Step 14: Why the hack reverts (and why we patch next)

Press **Resume**. The loop branches back to `0x1000264` , which reloads `movs r1, #42` , so the next line is `regular_fav_num: 42` . The live edit changed one iteration only. There is no stack variable in memory to change; the value is baked into the instruction. To make `regular_fav_num: 43` permanent we must patch the instruction. That is the static pass.

Press **Pause** to stop the output flood.

Step 15: Kill the debugger and OpenOCD

The live hack is done. Do this **before** the static pass.

1. In the **Debugger** sidebar, click the **X (Kill)** (or `Debugger -> Kill`) to disconnect IDA.
2. **Kill does not stop the OpenOCD process** — `debug-server.sh` started it separately, and it keeps running and holding the probe. Stop it from the IDA console:

macOS / Linux:

```
import subprocess
subprocess.run(["pkill", "-TERM", "-f", "openocd"]) # stop the debug server, free the probe
```

Windows:

```
import subprocess
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"]) # stop the debug server, free the probe
```

3. Confirm nothing is left: `pgrep -fl openocd` (macOS/Linux) prints nothing.

From a terminal it is the same: `pkill -TERM -f openocd`, or `Get-Process openocd | Stop-Process` on Windows.

Part 4: Static — Resolve the Functions in IDA and Patch

Step 16: Resolve the functions in the IDA GUI

We now name the functions in IDA using the ELF symbol map from Step 4. IDA loaded the raw `.bin` with **no symbols**, so every function shows as `sub_<addr>` — resolving means giving each one its real name and signature.

Three keys do all the work:

Key	IDA action	Use it for
G	Go to address	Jump to a function's address
Y	Change Type	Set the function's signature. The dialog shows the full prototype, so this sets the name <i>and</i> the type in one step.
N	Rename	Rename only, when you just want the name and not the type

For each function below: **G** to its address, then **Y (Change Type)** and type the prototype from the table.

How to resolve a function in IDA (Y)

Y is the **Change Type** key, and it is what actually resolves the function — it turns `void sub_10003014()` into `bool stdio_init_all(void)`. The Change Type dialog shows the full declaration (name and type), so typing the prototype sets both:

1. G to the function's address. The cursor lands on the function.
2. Press Y. In the Change Type dialog, type the prototype from the table exactly — for example `bool stdio_init_all(void)` — and press Enter.

The decompiler header then shows the real prototype, and calls to the function read cleanly instead of `sub_<addr>()`. N is only for renaming without touching the type; Y alone sets both the name and the type.

If Y seems to do nothing, confirm the cursor is on the function, or right-click it and pick **Change Type....** IDA parses what you type and silently keeps the old type if it does not parse, so glance at the header after each Y.

Worked example: `main`

1. Press G, type `0x10000234`, press Enter. The view jumps there; the cursor lands on `sub_10000234`.
2. Press Y (Change Type), type `int main(void)`, press Enter. That sets the name to `main` and the type to `int(void)`.

IDA shows `int32_t` where Ghidra shows `int`. After you set `int main(void)`, the decompiler header may read `int32_t main(void)`. That is the same type — on this platform `int` is 32 bits and IDA's parser normalises it to `int32_t`. Do not fight it; it is not an error.

Worked example: `gpio_init`

1. G -> `0x10000300`.
2. Y -> `void gpio_init(uint gpio)`.

Worked example: `gpio_set_pulls`

1. G -> `0x100002d8`.
2. Y -> `void gpio_set_pulls(uint gpio, bool up, bool down)`.

This is what `gpio_pull_up(15)` in our source compiles to: the SDK's `static inline gpio_pull_up` disappears, and `main` calls `gpio_set_pulls(15, true, false)` directly at `0x1000024e`.

Worked example: `gpio_set_function`

1. G -> `0x1000029c`.
2. Y -> `void gpio_set_function(uint gpio, gpio_function_t fn)`.

Worked example: `__wrap_printf`

1. G -> `0x100031a4`.
2. Y -> `int __wrap_printf(const char *fmt, ...)`. Keep the `...` — `printf` is variadic.

printf in our source is `__wrap_printf` in the binary. The SDK links our `printf` calls to its `__wrap_printf` wrapper.

The rest of the chain is the same two keystrokes per function (G, then Y). This is **our code plus the library functions it actually calls** — not the whole SDK. `main` calls `stdio_init_all`, `gpio_init`, `gpio_set_pulls`, and `printf`, so we follow that chain down: `stdio_init_all` pulls in the `stdio/UART` setup, `printf` lands in the SDK's `__wrap_printf`, and `uart_init` reaches `clock_get_hz` and `busy_wait_us`.

The call chain for this project:

```

main
├── stdio_init_all
│   └── stdio_uart_init — gpio_set_function, uart_init, stdio_set_driver_enabled
│       └── uart_init — clock_get_hz, busy_wait_us
├── gpio_init
├── gpio_set_pulls (gpio_pull_up inlined)
├── __wrap_printf — __wrap_vprintf
│   ├── vfctprintf — vsnprintf
│   │   └── _ntoa_format / _out_rev / _out_char
│   ├── stdio_out_chars_crlf
│   └── time_us_64
└── gpio_set_dir / gpio_get / gpio_put (inlined; no calls)

```

Resolve every function in that chain:

Address	Rename to (N)	Signature (Y)
0x1000015c	_reset_handler	void _reset_handler(void)
0x10000186	platform_entry	void platform_entry(void)
0x1000019a	data_cpy	void data_cpy(void*, void*, void*)
0x100001e4	_init	void _init(void)
0x10000210	frame_dummy	void frame_dummy(void)
0x10000234	main	int main(void)
0x1000029c	gpio_set_function	void gpio_set_function(uint, gpio_function_t)
0x100002d8	gpio_set_pulls	void gpio_set_pulls(uint, bool, bool)
0x10000300	gpio_init	void gpio_init(uint)
0x10000e60	time_us_64	uint64_t time_us_64(void)
0x10000ed0	uart_init	uint uart_init(uart_inst_t*, uint)
0x10002dbc	exit	void exit(int)
0x10002dc4	runtime_init	void runtime_init(void)
0x10002df0	stdio_out_chars_crlf	void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)
0x10002f00	stdio_put_string	int stdio_put_string(const char*, int, bool, bool)
0x10002fec	stdio_set_driver_enabled	void stdio_set_driver_enabled(stdio_driver_t*, bool)
0x10003014	stdio_init_all	bool stdio_init_all(void)
0x100030e0	__wrap_vprintf	int __wrap_vprintf(const char*, va_list)
0x100031a4	__wrap_printf	int __wrap_printf(const char*, ...)
0x10003360	stdio_uart_init	void stdio_uart_init(void)
0x100034a0	strlen	size_t strlen(const char*)

Resolve the SDK helpers the `printf` and `uart_init` paths reach:

Address	Rename to (N)	Signature (Y)
0x10002d60	<code>vfctprintf</code>	<code>int vfctprintf(void (*)(char, void*), void*, const char*, va_list)</code>
0x1000237c	<code>_vsprintf</code>	<code>int _vsprintf(out_fct_type, char*, size_t, const char*, va_list)</code>
0x10001760	<code>_ntoa_format</code>	<code>unsigned _ntoa_format(out_fct_type, char*, size_t, size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)</code>
0x100016c4	<code>_out_rev</code>	<code>unsigned _out_rev(out_fct_type, char*, size_t, size_t, const char*, size_t, unsigned, unsigned)</code>
0x10001934	<code>_out_char</code>	<code>void _out_char(char, void*, size_t, size_t)</code>
0x10000e74	<code>busy_wait_us</code>	<code>void busy_wait_us(uint64_t)</code>
0x100010a4	<code>clock_get_hz</code>	<code>unsigned long clock_get_hz(clock_handle_t)</code>

A `void` return type may not stick — here is the fix. IDA treats `void` as low-confidence, and its analysis can override it with an inferred type — most often `int32_t` on this 32-bit target. It is most visible on `_reset_handler` (a hand-written assembly entry that never returns normally), but it can happen to **any** function whose return type IDA thinks it can infer.

Setting the full signature with `Y` reproduces the unwanted `int32_t`, and `fn.return_type = ...` fails too. What works is the **return-value** setter:

```
from binaryninja import ReturnValue, Type
fn = bv.get_function_at(0x1000015c)
if fn is not None:
    fn.return_value = ReturnValue(Type.void())
```

That holds `_reset_handler` at `void` even after reanalysis. If it still will not stick, leave it — it does not affect the rest of the lesson.

`__wrap_printf` is the real symbol. `printf` in our source compiles to the SDK's `__wrap_printf` (which forwards to `__wrap_vprintf`). Rename it `printf` if you prefer the lesson's shorthand, but `__wrap_printf` is what the ELF says.

`stdio_init_all` **returns** `bool`, **not** `void` — `_Bool stdio_init_all(void)` in the ELF.

Shortcut — resolves name *and* type for every function. Instead of doing `N + Y` by hand, paste this into IDA's Python console (`Plugins -> Python Console`). It sets each function's name and signature programmatically:

```

from binaryninja import Symbol, SymbolType
# The raw .bin has no headers, so these SDK types don't exist. set_user_type()
# re-parses each signature as C, so an undefined name raises
# "SyntaxError: unknown type name '...'. Define them first.
sdk = bv.parse_types_from_string("""
typedef unsigned int uint;
typedef char* va_list;
typedef unsigned long clock_handle_t;
typedef void (*out_fct_type)(char, void*, size_t, size_t);
struct stdio_driver;
typedef struct stdio_driver stdio_driver_t;
struct uart_inst;
typedef struct uart_inst uart_inst_t;
enum gpio_function {
    GPIO_FUNC_XIP = 0, GPIO_FUNC_SPI = 1, GPIO_FUNC_UART = 2, GPIO_FUNC_I2C = 3,
    GPIO_FUNC_PWM = 4, GPIO_FUNC_SIO = 5, GPIO_FUNC_PI00 = 6, GPIO_FUNC_PI01 = 7,
    GPIO_FUNC_GPCK = 8, GPIO_FUNC_USB = 9, GPIO_FUNC_NULL = 0x1f,
};
typedef enum gpio_function gpio_function_t;
""")
for name, ty in sdk.types.items():
    bv.define_user_type(name, ty)

# address: (name, signature)
funcs = {
    0x1000015c: ("_reset_handler", "void _reset_handler(void)"),
    0x10000186: ("platform_entry", "void platform_entry(void)"),
    0x1000019a: ("data_cpy", "void data_cpy(void*, void*, void*)"),
    0x100001e4: ("_init", "void _init(void)"),
    0x10000210: ("frame_dummy", "void frame_dummy(void)"),
    0x10000234: ("main", "int main(void)"),
    0x1000029c: ("gpio_set_function", "void gpio_set_function(uint, gpio_function_t)"),
    0x100002d8: ("gpio_set_pulls", "void gpio_set_pulls(uint, bool, bool)"),
    0x10000300: ("gpio_init", "void gpio_init(uint)"),
    0x10000e60: ("time_us_64", "uint64_t time_us_64(void)"),
    0x10000ed0: ("uart_init", "uint uart_init(uart_inst_t*, uint)"),
    0x10002dbc: ("exit", "void exit(int)"),
    0x10002dc4: ("runtime_init", "void runtime_init(void)"),
    0x10002df0: ("stdio_out_chars_crlf", "void stdio_out_chars_crlf(stdio_driver_t*, const
char*, int)"),
    0x10002f00: ("stdio_put_string", "int stdio_put_string(const char*, int, bool,
bool)"),
    0x10002fec: ("stdio_set_driver_enabled", "void stdio_set_driver_enabled(stdio_driver_t*,
bool)"),
    0x10003014: ("stdio_init_all", "bool stdio_init_all(void)"),
    0x100030e0: ("__wrap_vprintf", "int __wrap_vprintf(const char*, va_list)"),
    0x100031a4: ("__wrap_printf", "int __wrap_printf(const char*, ...)"),
    0x10003360: ("stdio_uart_init", "void stdio_uart_init(void)"),
    0x100034a0: ("strlen", "size_t strlen(const char*)"),
    0x10002d60: ("vfctprintf", "int vfctprintf(void (*)(char, void*), void*, const
char*, va_list)"),
    0x1000237c: ("_vsprintf", "int _vsprintf(out_fct_type, char*, size_t, const
char*, va_list)"),
    0x10001760: ("_ntoa_format", "unsigned _ntoa_format(out_fct_type, char*, size_t,
size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)"),
    0x100016c4: ("_out_rev", "unsigned _out_rev(out_fct_type, char*, size_t,
size_t, const char*, size_t, unsigned, unsigned)"),
    0x10001934: ("_out_char", "void _out_char(char, void*, size_t, size_t)"),
    0x10000e74: ("busy_wait_us", "void busy_wait_us(uint64_t)"),
    0x100010a4: ("clock_get_hz", "unsigned long clock_get_hz(clock_handle_t)"),
}
for addr, (name, sig) in funcs.items():
    bv.define_user_symbol(Symbol(SymbolType.FunctionSymbol, addr, name))
    f = bv.get_function_at(addr)
    if f is not None:
        f.set_user_type(sig)

```

SDK type names (`stdio_driver_t`, `gpio_function_t`, `uart_inst_t`, plus `uint`, `va_list`, `out_fct_type`, and `clock_handle_t`) are **not** in the raw `.bin`. `set_user_type` re-parses each signature as C, so an undefined name raises `SyntaxError: unknown type name '...'` and stops the loop — it is not harmless. The `sdk` block above defines them first (an opaque `struct` / `enum` / `typedef` is enough to parse). If you add a function that uses another SDK type, add a definition for it to that block too.

Step 17: Read `main` in the decompiler

Open the **Decompiler** view on `main`. Once the functions above are typed, it reads roughly:

```
int32_t main(void)
{
    stdio_init_all();
    gpio_init(0xf);
    gpio_set_dir(0xf, GPIO_IN);           // inlined: mcr r0, 4, r0, r3, cr4
    gpio_set_pulls(0xf, true, false);     // gpio_pull_up(15)
    gpio_init(0x10);
    gpio_set_dir(0x10, GPIO_OUT);         // inlined
    do
    {
        __wrap_printf("regular_fav_num: %d\r\n", 0x2a);
        __wrap_printf("static_fav_num: %d\r\n", *(uint8_t*)0x200005a8);
        *(uint8_t*)0x200005a8 = *(uint8_t*)0x200005a8 + 1; // static_fav_num++
        gpio_put(0x10, gpio_get(0xf) ^ 1);                // inlined SI0 read/write
    } while (true);
}
```

The `0x2a` is the value we edited live; `0x200005a8` is the fixed RAM address of `static_fav_num`. The `gpio_get(0xf) ^ 1` is the `eor.w r3, r3, #1` we will patch in Step 18b. Now make the hacks permanent.

Step 18: Patch 1 — change `regular_fav_num` from 42 to 43

Go to `0x10000264`:

```
10000264: 212a  movs  r1, #42 ; 0x2a
```

The halfword is `0x212a`, stored little-endian as `2a 21`. The immediate is the low byte, so the byte at the instruction's own address is `0x2a`. Change it to `0x2b` (43).

Project	Address	Before	After	Effect
0x0014	0x10000264	2a	2b	<code>movs r1, #42 -> #43</code> , prints <code>regular_fav_num: 43</code>

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Toggle the lock off so editing is enabled.
3. Go to `0x10000264` and change the byte `2A` to `2B`.
4. Return to the linear view, right-click the function -> `Reanalyze`.

Option B — Python console:

```
bv.write(0x10000264, b"\x2b")
print(hex(bv.read(0x10000264, 1)[0])) # -> 0x2b
```

After reanalysis the instruction reads `movs r1, #43`.

Step 18b: Patch 2 — invert the button/LED logic

The stock code inverts the raw button bit with `eor.w r3, r3, #1` at `0x10000286`. Its four bytes are `83 f0 01 03`; the immediate `#1` is the **third** byte, at `0x10000288`. Change `01` to `00` so the XOR becomes `#0` (a no-op) and the LED follows the raw pin state instead:

```
10000286: f083 0301 eor.w r3, r3, #1 ; immediate byte at 0x10000288
```

Project	Address	Before	After	Effect
0x0014	0x10000288	01	00	<code>eor.w r3, r3, #1</code> -> <code>#0</code> , inverts the LED behavior

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Go to `0x10000288` and change the byte `01` to `00`.
3. Return to the linear view and reanalyze.

Option B — Python console:

```
bv.write(0x10000288, b"\x00")
print(bv.read(0x10000286, 4).hex()) # -> 83f00003
```

Now the logic is permanently changed:

- Button released (GPIO 15 reads `1`): `1 XOR 0 = 1` -> LED **ON**
- Button pressed (GPIO 15 reads `0`): `0 XOR 0 = 0` -> LED **OFF**

This is the **opposite** of the original behavior. The `gpio_get` and `gpio_put` are inlined, so the only byte that controls the inversion is this immediate.

Step 18c: Patch 3 — rename the printed label (optional)

The format string `"regular_fav_num: %d\r\n"` starts at `0x10003560`. Its first 16 bytes are `72 65 67 75 6c 61 72 5f 66 61 76 5f 6e 75 6d 3a` (`regular_fav_num:`). Change them to `70 61 74 63 68 65 64 5f 66 61 76 5f 6e 75 6d 3a` (`patched_fav_num:`), leaving the `%d\r\n` tail untouched, so the line prints `patched_fav_num: 43`.

Project	Address	Before	After	Effect
0x0014	0x10003560	72 65 67 75 6c 61 72 5f 66 61 76 5f 6e 75 6d 3a	70 61 74 63 68 65 64 5f 66 61 76 5f 6e 75 6d 3a	<code>regular_fav_num:</code> -> <code>patched_fav_num:</code>

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Go to `0x10003560` and change the sixteen bytes `72 65 67 75 6c 61 72 5f 66 61 76 5f 6e 75 6d 3a` to `70 61 74 63 68 65 64 5f 66 61 76 5f 6e 75 6d 3a`.
3. Return to the linear view and reanalyze.

Option B — Python console:

```

bv.write(0x10003560, b"patched_fav_num:")
print(bv.read(0x10003560, 22)) # -> b'patched_fav_num: %d\r\n\x00'

```

Keep the replacement exactly sixteen bytes — the same length as `regular_fav_num:`. If you use a shorter string you must pad it, or `%d` shifts and `printf` reads the wrong argument. A longer string would overwrite the `%d` tail.

Step 19: Export the patched .bin

```

import os
seg = next(s for s in bv.segments if s.data_length) # the loadable image segment
data = bv.read(seg.start, seg.data_length) # base + size come from the view itself
out = os.path.join(os.path.join(root, "0x0014_static-variables", "build"), "0x0014_static-variables-h.bin")
open(out, "wb").write(data)
print(len(data), out) # -> 15516 ../../build/0x0014_static-variables-h.bin

```

Where the two numbers come from — nothing is hardcoded:

- `seg.start` is the image base IDA loaded the `.bin` at (`0x10000000`), the same value you pass to `uf2conv --base`.
- `seg.data_length` is the segment's size in the file (`0x3c9c = 15516`). Exactly one segment carries data (the image); every peripheral and synthetic segment has `data_length == 0`, so `next(...)` picks the image.
- Reading `seg.start` for `seg.data_length` bytes therefore grabs exactly the image.

Two gotchas this avoids:

- **No relative path.** IDA's Python console runs with a read-only working directory (inside the app bundle), so `open("0x0014_static-variables-h.bin", "wb")` fails with `OSError: [Errno 30] Read-only file system`. `root` (from `~/embedded-hacking-repo`, Step 3) is the repo, so the file is written into the project's `build/` — no machine-specific path and no database needed.
- **Read the image, not the whole view.** `bv.read(bv.start, bv.length)` spans the entire mapped range, which is not the image. The segment's `data_length` is the image size.

A different size means you exported a partial view.

Step 20: Convert to UF2

Run from the project directory:

macOS Apple Silicon / Linux x64:

```

python3 ../uf2conv.py 0x0014_static-variables-h.bin \
--base 0x10000000 --family 0xe48bff59 --output hacked.uf2

```

Windows x64:

```

python ../uf2conv.py 0x0014_static-variables-h.bin ^
--base 0x10000000 --family 0xe48bff59 --output hacked.uf2

```

Or convert from the IDA console — it is a normal Python interpreter, so you never have to leave the app. `chdir` to a writable directory first (the default one is read-only), then run the script:

```
import os, sys, runpy
os.chdir(os.path.join(root, "0x0014_static-variables", "build")) # the project build dir
(writable)
sys.argv = ["uf2conv.py", "0x0014_static-variables-h.bin",
            "--base", "0x10000000", "--family", "0xe48bff59", "--output", "hacked.uf2"]
runpy.run_path("../uf2conv.py", run_name="__main__") # path to your uf2conv.py
```

This writes `hacked.uf2` next to the `.bin`, ready to drag onto the Pico.

Step 21: Flash and verify

Hold **BOOTSEL**, plug in the Pico 2, and drag `hacked.uf2` onto the `RP2350` drive. Open the serial monitor:

```
patched_fav_num: 43
static_fav_num: 42
patched_fav_num: 43
static_fav_num: 43
patched_fav_num: 43
static_fav_num: 44
...
```

and the **LED is on by default** (button released) and turns **off** when you press the button — the opposite of the stock image.

The regular value is now 43, the label reads `patched_fav_num:`, and the button/LED logic is inverted — with eighteen bytes changed and no source code. (`static_fav_num` keeps incrementing from 42, unchanged: we left the static variable itself alone so you can watch it persist.)

Faster: flash over the Debug Probe (no BOOTSEL). The repo's `flash.sh` writes the raw `.bin` straight into XIP flash over SWD (`program <bin> 0x10000000 verify reset exit`), so you never touch BOOTSEL or a UF2. Run it from a terminal (`./flash.sh <bin>`), or from the IDA console **without freezing it** — use `subprocess.Popen`, which returns immediately, and send OpenOCD's output to a log file. (`subprocess.run` blocks the console until the flash finishes; do not use it here.)

```
import os, subprocess
root = open(os.path.expanduser("~/embedded-hacking-repo")).read().strip()
bin_path = os.path.join(os.path.join(root, "0x0014_static-variables", "build"), "0x0014_static-variables-h.bin")
log = os.path.join(os.path.join(root, "0x0014_static-variables", "build"), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"]) # free the probe first
p = subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                     stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

The `pkill` frees the probe first; on Windows use `subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])`.

The console is free the moment this returns. Check it with `print(p.poll())` (`None` = still running, `0` = done) or read `flash.log` — success ends with `** Verified OK **`.

The same non-blocking form without the script:

```
import os, subprocess
ocd = os.path.expanduser("~/pico-sdk/openocd/0.12.0+dev")
bin_path = os.path.join(os.path.join(root, "0x0014_static-variables", "build"), "0x0014_static-variables-h.bin")
log = os.path.join(os.path.join(root, "0x0014_static-variables", "build"), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"]) # free the probe first
p = subprocess.Popen([f"{ocd}/openocd", "-s", f"{ocd}/scripts",
                     "-f", "interface/cmsis-dap.cfg", "-f", "target/rp2350.cfg",
                     "-c", "adapter speed 5000",
                     "-c", f"program {bin_path} 0x10000000 verify reset exit"],
                     stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

The Debug Probe is single-owner. If IDA is still attached (the `debug-server.sh` OpenOCD is running), the flash cannot grab the probe. Detach in IDA and stop that OpenOCD first:

```
# macOS / Linux
pkill -TERM -f openocd
```

```
# Windows
Get-Process openocd -ErrorAction SilentlyContinue | Stop-Process
```

Success looks like `Programming Finished -> Verified OK -> Resetting Target`. On Windows use `flash.ps1` (`.\flash.ps1 -Bin <path>`) the same way.

Cheatsheet

IDA GUI actions

Action	How
Go to address	<code>G</code>
Rename function/symbol	<code>N</code>
Set type or signature	<code>Y</code>
Add comment	<code>;</code>
Open Hex view	<code>View -> Hex</code>
Enable hex editing	Toggle the lock in the status bar
Reanalyze after a patch	Right-click function -> Reanalyze
Edit a register live	<code>dbg.set_reg_value("r1", 0x2b)</code> in the Python console (or right-click the register, press <code>E</code> , type hex, Enter)
Write a RAM string live	<code>dbg.write_memory(0x20080000, b"patched_fav_num: %d\r\n\x00")</code>
Set a breakpoint	Debugger -> Add Hardware Breakpoint... (hardware execute). Do not use <code>F2</code> — software breakpoints cannot be written to read-only flash.
Move a breakpoint	Remove it and set it at the new address in the GUI (command-port fallback: <code>rbp <old addr></code> then <code>bp <new addr> 2 hw</code>)
Confirm what is armed	The Breakpoints widget lists it (command-port fallback: <code>mdw 0xE0002000 8</code> , each armed breakpoint shows as <code><addr 1></code>)
Apply the ELF symbol map	Paste the Python snippet from Step 16 into the Python Console

OpenOCD server and reset

The server runs with `gdb_breakpoint_override hard` so that flash-writes are never attempted. Breakpoints in this lab are set in the IDA GUI through the **GDB MI** adapter (Step 12). The command-port rows below are the fallback if you use the **GDB RSP** adapter instead.

Action	Command
Connect to the OpenOCD prompt (fallback)	<code>nc 127.0.0.1 4444</code> (or <code>telnet 127.0.0.1 4444</code>)
Reset and run (command port)	<code>reset run</code>
Check core state (command port)	<code>targets</code>

Action	Command
Set a breakpoint in the GUI	Debugger -> Add Hardware Breakpoint... (hardware execute; F2 software breakpoints do not work on flash)
(fallback) Add a breakpoint without the GUI	<code>bp <addr> 2 hw</code>
Remove one breakpoint	<code>rbp <addr> — address only, no length, no hw</code>
Remove every breakpoint	<code>rbp all</code>
Start the server parked at <code>main</code>	macOS/Linux: <code>BP_ADDR=0x10000234 ./debug-server.sh</code> — Windows: <code>\$env:BP_ADDR="0x10000234"; .\debug-server.ps1</code> (one-shot)
Start the server parked in the loop	macOS/Linux: <code>BP_ADDR=0x10000268 ./debug-server.sh</code> — Windows: <code>\$env:BP_ADDR="0x10000268"; .\debug-server.ps1</code>
Break on the loop in a running target	set a hardware breakpoint in the GUI at the loop address, then Resume — repeatable
Make IDA stepping work	<code>rp2350.dap.core0 configure -rtos none</code> (already in the scripts)
Step without re-trapping	move the breakpoint off the current PC first, then Step Into/Step Over
Reset without desyncing IDA	Detach , <code>reset run</code> on the port, reconnect — never <code>reset run</code> while attached

Every address and byte we changed

Project	Address	Before	After	Effect
0x0014	0x10000264	2a	2b	<code>movs r1, #42 -> #43</code> , prints <code>regular_fav_num: 43</code>
0x0014	0x10000288	01	00	<code>eor.w r3, r3, #1 -> #0</code> , inverts the button/LED logic
0x0014	0x10003560	72 65 67 75 6c 61 72 5f 66 61 76 5f 6e 75 6d 3a	70 61 74 63 68 65 64 5f 66 61 76 5f 6e 75 6d 3a	string prints <code>patched_fav_num:</code> instead of <code>regular_fav_num:</code>

The static-variable memory map

Item	Address	Notes
<code>static_fav_num</code>	0x200005a8	RAM <code>.data</code> , fixed for the life of the program; initial value 42 copied from flash at boot
Literal-pool word 1	0x10000290	0x200005a8 — the RAM address <code>r4</code> holds
Literal-pool word 2	0x10000294	0x10003560 — pointer to <code>"regular_fav_num: %d\r\n"</code>
Literal-pool word 3	0x10000298	0x10003578 — pointer to <code>"static_fav_num: %d\r\n"</code>
<code>regular_fav_num</code>	(no address)	inlined to <code>movs r1, #42</code> ; never given a stack slot

Raw image facts

Item	Value
Build type	Release
Load base address	0x10000000
Project size	15516 bytes (0x3c9c)
Fixed main anchor	0x1000018c (reset handler middle blx)
main	0x10000234
printf call / return, regular_fav_num	0x10000268 / 0x1000026c
printf call, static_fav_num	0x10000270
static_fav_num RAM address	0x200005a8
regular_fav_num format string	0x10003560
static_fav_num format string	0x10003578
RP2350 UF2 family ID	0xe48bff59

Troubleshooting

IDA hangs or crashes when you connect (macOS 27)

Three different causes have been seen on this setup; check them in this order.

- **A breakpoint set before connecting.** With the **GDB MI** adapter, if the binary view already has a breakpoint, the session hangs. Start parked with `BP_ADDR`, connect, then add breakpoints (see the next entry).
- **The wrong GDB executable.** Point **Full GDB Executable Path** at the **14.2.rel1** toolchain (Step 10). The 13.3.rel1 build did **not** connect in testing.
- **The LLDB adapter.** A crash report with `libdebuggercore.dylib -> std::terminate() -> abort()` and `liblldb` in the stack is the **LLDB** adapter, not GDB MI. Avoid LLDB on this setup.

Use **GDB MI**, with the 14.2.rel1 path above. If it still fails, fall back to plain `arm-none-eabi-gdb` against the same server — the addresses and register values are identical to the GUI steps.

If IDA hangs, force-quit it; the connect dialog has no working Cancel. The static steps (resolve, patch, export, flash) never touch the debugger and always work.

The GUI refuses to set a breakpoint (GDB RSP adapter only)

If you are on the **GDB RSP** adapter, the GUI cannot set breakpoints on this target. That adapter is IDA's own minimal RSP client and sends a **1-byte** breakpoint (`Z0,<addr>,1`); the Cortex-M33 comparators need 2 bytes, so OpenOCD answers only breakpoints of two bytes length supported. It affects every address, both **Toggle Breakpoint** and **Add Hardware Breakpoint**, and the dialog's **Size** field is disabled. `gdb_breakpoint_override` makes no difference.

Fix: use the GDB MI adapter (Step 10). It drives real GDB, which sends the correct length, so GUI breakpoints just work. If you must stay on GDB RSP, arm breakpoints from the command port after connecting (`bp <addr> 2 hw`) — but the lab uses GDB MI and does not need that.

GDB MI hangs when you connect (a breakpoint already existed)

With the **GDB MI** adapter, if IDA already has a breakpoint set when you connect, the session **hangs**. This is a IDA bug. The working order is:

1. Start the server parked, e.g. `BP_ADDR=0x10000234 ./debug-server.sh` (Windows: `$env:BP_ADDR="0x10000234"; .\debug-server.ps1`).
2. Connect with the **GDB MI** adapter.
3. Only *then* set hardware breakpoints in the UI.

Never have a breakpoint in the binary view before the GDB MI connection. If it hangs, quit IDA, restart the server with `BP_ADDR` , and connect again before adding any breakpoints.

Step Into / Step Over does nothing (PC never moves)

Two causes have been seen on this target.

1. **A breakpoint on the current PC re-traps the step.** OpenOCD's step-over-breakpoint logic fails with `Duplicate Breakpoint address` and the PC stays put. Fix: move the breakpoint off the current PC (in the GUI), then step.
2. **The hwthread RTOS (GDB RSP adapter only).** With the **GDB RSP** adapter, OpenOCD can log `fake step thread 0` and reply without stepping, because the RP2350 config's `-rtos hwthread` makes the current thread id 1 while IDA sends thread id 0. Fix: `rp2350.dap.core0 configure -rtos none` (the launcher scripts already pass this). **GDB MI does not hit this.**

To tell them apart, turn on OpenOCD logging (`log_output /tmp/ocd.log` , then `debug_level 3` on the command port) and look for `fake step` versus `Duplicate Breakpoint` .

```
zsh: bad CPU type in executable: cmake
```

An Intel `x86_64` tool is on your `PATH` on Apple Silicon. Run Step 2: `export PATH="/opt/homebrew/bin:$PATH"` , then `hash -r` . Add it to `~/.zshrc` to make it permanent.

My addresses do not match this guide

You probably built `Debug` . This lesson is a `Release` build. Re-run Step 3 with `-DCMAKE_BUILD_TYPE=Release` . A `Debug` build moves the SDK functions and keeps `demo_static_variable` separate, so `main` is not at `0x10000234` .

A breakpoint never fires

First, confirm you actually set one, and that it is a **hardware** breakpoint. With the **GDB MI** adapter, `Debugger -> Add Hardware Breakpoint...` (hardware execute) should land in the **Breakpoints** widget. If nothing lands, or the core keeps running, you probably used `F2 (Toggle Breakpoint)` — that is a software breakpoint and cannot be written to read-only flash, so it never installs. Also check you are on **GDB MI**, not **GDB RSP** (the GDB RSP adapter cannot set breakpoints on this target at all).

Then check the order and the state:

- **Arm it only after IDA is connected.** OpenOCD flushes every breakpoint when a client attaches, so anything armed earlier is gone. This also applies to `BP_ADDR` on the startup command line.

- **Verify it is armed:** `mdw 0xE0002000 8`. You should see your address with the low bit set (`0x10000234` -> `0x10000235`). All zeros means nothing is armed — re-read this first, because it distinguishes "not armed" from "armed but never reached".
- **Is the core running?** `poll` on the command port should not report a halt. If it is stopped, click **Resume**.
- **Does the address get reached again?** `main` runs once per reset, so use `BP_ADDR` at startup (Step 9) rather than `reset run` while attached. Loop addresses such as `0x10000268` fire on the next pass with no reset — arm them and click **Resume** in IDA.
- **With GDB MI the stop is reported as** `Breakpoint` and appears in the **Breakpoints** widget, because GDB really did set it.

I edit `r1` (or another register) and it reverts

`main` reloads the value at the top of every loop iteration — `movs r1, #42` at `0x10000264` runs right before the `printf` at `0x10000268`. So `r1` is only `0x2b` for the instant between your edit and the next pass; then it is `0x2a` again. The edit sticks only if the core is **genuinely stopped** at the breakpoint and stays stopped.

If it keeps reverting, the core is running, which almost always means the breakpoint is not installed — usually because it is a **software** breakpoint (`F2`) that cannot be written to read-only flash. Use `Debugger -> Add Hardware Breakpoint...` (hardware execute).

The Registers widget is a snapshot, not a live view. IDA reads the registers at each stop and shows that snapshot; it does not poll the target, and there is no "refresh registers" command. So a value changed outside IDA will not appear until the next stop.

The static variable shows a wrong value in GDB / IDA

`static_fav_num` is a function-local `static`, so it does not appear in the global symbol table as `static_fav_num`; `arm-none-eabi-nm` names it `static_fav_num.0`. A 4-byte read at `0x200005a8` returns `0x0000122a`, whose low byte `0x2a` (42) is the variable — the upper bytes belong to whatever is in adjacent RAM. Read **one byte** (`x/1ub 0x200005a8`) to see `42`, and never add a `*` (that would treat the value as a pointer). In IDA, define a `uint8_t` at `0x200005a8` if you want the decompiler to show the byte cleanly.

The serial capture is garbage on macOS

Reading `/dev/cu.usbmodem*` with a bare `read()` returns garbage. Set **raw termios at 115200** first: clear canonical/echo flags, set `CLOCAL|CREAD`, and `B115200` on input and output. `screen /dev/cu.usbmodem* 115200` does all of this for you; a script must call `tcsetattr` itself. Once set, the capture reads clean `regular_fav_num`: 42 lines.

It worked for a second, then stopped (IDA's view desyncs)

This is the most common failure, and it has one main cause: **driving the core from the OpenOCD command port while IDA is connected.**

- If you send `reset run` from the port while attached, the core resets, runs, and halts at your breakpoint — but IDA never receives the stop event. Its sidebar keeps showing the *previous* location, so **Step** and **Resume** act on a stale PC and appear to do nothing.
- If the OpenOCD process dies (or you restart it) while attached, IDA keeps believing it is connected: the sidebar stays, but the menu shows **Pause** enabled and **Resume/Step** disabled because IDA last saw the target *running*.

Recovery: **Detach, then reconnect.** If Detach does nothing (the connection is already dead), restart IDA — its menu still shows a session that no longer exists.

Prevention:

- Stop at `main` with `BP_ADDR` on a fresh server start, not with `reset run` while attached.
- For loop addresses, set the breakpoint in the GUI and click **Resume**. Let IDA be the thing that starts the core.
- If you must reset, **Detach first**, `reset run`, then reconnect.
- Never leave a breakpoint on the PC you are about to step or resume from.

The target "blows past" `main` and stops at `0x100032cc` instead

`0x100032cc` is inside `stdio_uart_out_flush`:

```
100032c8: 4b02    ldr r3, [pc, #8] ; @ 0x100032d4
100032ca: 681a    ldr r2, [r3]
100032cc: 6993    ldr r3, [r2, #24] ; the core sits here while the UART drains
100032ce: 071b    lsls r3, r3, #28
100032d0: d4fc    bmi.n 0x100032cc
100032d2: 4770    bx lr
100032d4: 20000850 .word 0x20000850
```

That is the UART transmit-FIFO drain loop inside `printf`, so the core is running `main`'s loop and simply spends nearly all its time there. The breakpoint at `main` did not fire because `main`'s entry runs exactly **once per reset**. If you arm the breakpoint after the reset, or set it while the target is already running and just resume, the core is already past `main` and will never re-execute it. Either arm the breakpoint **before** resetting, or break inside the loop at `0x10000268`, which fires every iteration.

`0x100032cc` **is not a function**. It is one instruction inside `stdio_uart_out_flush`, which starts at `0x100032c8`. If IDA has created a function at `0x100032cc` (for example because the debugger stopped at that PC), the decompiler shows garbage. Delete that bogus function (right-click it -> `Delete Function`, or put the cursor on it and press `U` to undefine) and reanalyze. The real function is `stdio_uart_out_flush` at `0x100032c8`.

The console floods with Failed to read memory at `0xf0000000`

Core1 is exposed. The scripts must run with `USE_CORE=0`. Stop the server, confirm only `core0` is reported, restart, then restart IDA.

`Connect to Remote Process` **is greyed out and Pause does nothing**

IDA is in a stale session, usually because the debug server restarted while attached. Quit and reopen IDA (or the `.i64`) and connect again.

The decompiler still shows the old value after patching

Right-click the function and choose `Reanalyze`.

The LED does not change after patching

You patched the wrong byte. The inversion immediate is the **third** byte of the 4-byte `eor.w` instruction: the instruction is at `0x10000286`, so the byte to change is `0x10000288`. Confirm it now reads `00`, not `01`.

Fallback: do the dynamic steps with GDB (macOS 27)

If IDA's debugger crashes on attach on macOS 27 (see Troubleshooting), you can still do the live hack with the ARM GDB from the toolchain, against the same OpenOCD server. The addresses and register values are identical to the GUI steps.

Start the debug server (Step 9), then in a new terminal:

```
arm-none-eabi-gdb
```

At the `(gdb)` prompt:

```
set architecture armv8-m.main
target extended-remote :3333
hbreak *0x10000268
continue
```

Do **not** run `monitor reset run` before `hbreak`. `0x10000268` is inside `main`'s loop, so the breakpoint fires on the next iteration with no reset. If you reset first, the core runs `main` and you will not catch it.

GDB stops at the `printf` call. Confirm the value, change it, and let it run:

```
info registers pc r1 # pc = 0x10000268, r1 = 0x2a
set $r1 = 0x2b
stepi
continue
```

The serial monitor prints `regular_fav_num: 43` for the iteration you changed — the same temporary live hack as editing `r1` in the IDA Registers widget. When you are done, press `Ctrl-C`, then `detach` and `quit`.

If you specifically want to stop at `main` (`0x10000234`), remember its entry runs only once per reset, so the breakpoint must be armed *before* the reset:

```
monitor reset halt
hbreak *0x10000234
continue
```

If you instead set it while the target is running and just `continue`, you will "blow past" `main` and catch the core inside `printf` — in this build at `0x100032cc`, the `stdio_uart_out_flush` UART-drain loop.

`hbreak` sets a hardware breakpoint, which is required for read-only flash. It works from plain GDB because GDB sends the 2-byte length the Cortex-M33 comparators need. IDA's **GDB MI** adapter goes through the same GDB, so its GUI breakpoints work too; the old **GDB RSP** adapter was the one that sent a 1-byte length and could not set breakpoints here.

Glossary

Term	Definition
<code>.bss</code>	Section for uninitialized (or zero-initialized) static/global variables; zeroed by startup code

Term	Definition
<code>.data</code>	Section for initialized static/global variables; copied from flash to SRAM at boot
<code>.elf</code>	Linked image with the symbol table; the ground truth for addresses and names
<code>.rodata</code>	Read-only section for constants and string literals; stays in flash
Automatic variable	A local variable created and destroyed with its block; lives on the stack (or is optimized away)
<code>eor</code> / XOR	Exclusive OR — flips bits where the operands differ
GPIO	General Purpose Input/Output — controllable pins on the microcontroller
Hardware breakpoint	A breakpoint serviced by the CPU comparators, required for read-only flash
Inlining	The optimizer replacing a function call with the function body; why <code>demo_static_variable</code> disappears
Literal pool	A block of 32-bit constants that Thumb-2 code reaches with PC-relative <code>ldr</code>
MMIO	Memory-mapped I/O — hardware registers accessed as memory addresses
Pull-up / pull-down	A resistor that holds an input pin at a defined level when nothing drives it
SIO	Single-cycle I/O — the fast GPIO block in the RP2350, at <code>0xd0000000</code>
Static variable	A variable with static storage duration; persists for the whole program and keeps a fixed address
Ternary operator	<code>condition ? value_if_true : value_if_false</code>
Thumb bit	Bit 0 of a Cortex-M function pointer; selects Thumb instruction mode
<code>ubfx</code>	Unsigned Bit Field Extract — pulls a bit field out of a register
UF2	USB Flashing Format — the file format the Pico 2 bootloader accepts
Vector table	The first words of flash: initial stack pointer and exception vectors

Remember: the ELF tells you what every address is, and the `.bin` is what you actually patch. Prove the behavior dynamically, read `r1` at the `printf` call, resolve the names from the ELF, then patch the bytes — the constant `42`, the inversion immediate, and (optionally) the label — and flash.